HOBBY PROJECT CASE STUDY

Software Server Queue Simulation Dashboard

An interactive queueing simulator that turns request traffic, service time, and worker capacity into visible trade-offs in waiting, response time, and SLA breaches.

SYSTEM

Interactive Simulation Dashboard

SCOPE

Workloads · Queues
Metrics · Comparisons

APPROACH

Seeded Experiments
Local-First Learning

01 The Problem

Queueing formulas describe capacity, but they do not make the experience of congestion easy to see. A small change in arrival rate or worker count can turn a short wait into a growing backlog.

For a hobby project exploring queueing models for software systems, the challenge was to connect abstract ideas to a repeatable experiment: how do traffic, service duration, and parallel workers shape the requests waiting for a backend?

> The dashboard brings controls, discrete-event simulation, visual evidence, and a capacity comparison into one local application. It is a learning tool, not live infrastructure monitoring or an automatic scaling service.

02 What Was Built

A self-contained simulation workspace for exploring an M/M/c-style queue and comparing the consequences of different worker counts.

WORKLOAD CONTROLS

Tune arrival rate, service time, worker count, duration, SLA threshold, and seed. Healthy, Overloaded, and Optimized presets provide quick starting points.

REQUEST-LEVEL SIMULATION

Follow each request from arrival through a shared worker pool to completion. Retain waiting, service, and response times for inspection.

METRICS & VISUAL EXPLANATION

Read seven KPIs alongside queue history, a response-time histogram, bottleneck status, and a utilization-based server recommendation.

REPRODUCIBLE COMPARISONS

Reuse one generated workload for the current and recommended worker counts. Inspect the comparison table, chart, and request event log.

03 How It Works

Streamlit collects the controls. NumPy generates a fixed workload, SimPy processes it through parallel workers, and pandas turns the event records into metrics. Plotly presents the results.

> **Controls → Seeded workload → SimPy worker pool → Metrics & charts**
> One workload → Current worker count + Recommended worker count
> Submitted form → Active session parameters → Updated analysis

Deterministic workload generation

A NumPy random generator samples exponential interarrival intervals and service durations. The seed fixes the generated request IDs, arrival times, and service times. Both scenarios receive that same workload, isolating the effect of changing capacity.

Requests, workers, and the drain period

A SimPy Resource limits concurrent service to the selected worker count. Requests wait when all workers are busy. Arrivals stop at the configured duration, but the simulation drains every arrived request; observed time can therefore extend beyond the arrival window.

Metrics with explicit denominators

Waiting time runs from arrival to service start; response time runs from arrival to completion. Average queue length integrates queue size over the full observed horizon. Throughput divides all completions by the configured arrival duration, not the drain horizon.

Capacity recommendation, not an SLA guarantee

Utilization is calculated from arrival rate, mean service time, and worker count. The recommendation selects the smallest count at or below 85% utilization, up to 10 workers. It does not minimize latency, enforce an SLA, or provision real infrastructure.

Model boundary. This is a simplified M/M/c-style experiment. It does not model network latency, database locks, caching, failures, or multiple request classes.

04 Key Features

Quick workload presets — Healthy, Overloaded, and Optimized scenarios populate every control, including seed 42. Values remain editable before Run Simulation applies them.

Controlled experiment workflow — A sidebar form separates pending edits from active results. Submitted parameters live in the Streamlit session; initial load uses the default scenario.

Seven connected KPIs — Completed requests, average waiting and response times, time-weighted queue length, theoretical utilization, throughput, and SLA breach rate describe the same run.

Event-aware queue history — Queue changes are recorded at simulation events and displayed as a step chart rather than suggesting gradual changes between events.

Response-time distribution — A histogram shows the spread of request response times with a visible SLA threshold; a breach is a response strictly above that threshold.

Bottleneck classification — Utilization bands distinguish spare capacity, good utilization, high risk, and overload. Offered load may exceed 100%; it is not measured busy-time utilization.

Baseline versus recommended capacity — A comparison table and grouped bar chart expose waiting-time, response-time, SLA, and utilization trade-offs using identical request demand.

Inspectable request event log — The first 50 completed-request records show arrivals, service starts, completions, waiting times, and service durations, rounded to two decimals.

Local and self-contained — Once dependencies are installed, the application needs no external APIs, databases, cloud services, authentication, or internet access.

05 Technical Approach

The project keeps simulation, analysis, chart creation, and rendering in one Python module, with separate functions for each responsibility. The stack stays small enough for a local hobby experiment.

LAYER	STACK	PURPOSE
Application	Python; Streamlit	Sidebar form, session state, KPI cards, and local dashboard delivery
Workload generation	NumPy random generator	Seeded exponential arrivals and service durations
Simulation engine	SimPy Environment / Resource	Discrete events, bounded parallel workers, and queued requests
Analysis	pandas; Python math	Request tables, weighted queue averages, metrics, and recommendations
Visualization	Plotly graph objects / express	Queue history, response distribution, and capacity comparison
Verification	pytest; pytest-cov	Model, metric, scaling, chart, and analysis smoke coverage

Design decisions that matter

Generate once, compare fairly. Sampling demand before simulation prevents a worker-count change from also changing the workload being compared.

Drain rather than discard. Letting existing requests finish preserves their eventual waiting and response times, even when the arrival window ends with a backlog.

Keep the heuristic visible. The 85% utilization target is a capacity rule, not a claim that the recommended configuration satisfies the selected SLA.

Repository guide: app.py holds the application; tests/ covers workload, simulation, metrics, scaling, charts, and smoke behavior; README.md documents setup and demo scenarios. Runtime dependencies are Streamlit, SimPy, pandas, NumPy, and Plotly.

06 Value Delivered

For a hobby project, the value is a concrete way to explore queueing behavior, reproduce a result, and question a capacity recommendation—not a claimed production or commercial outcome.

Visible

QUEUEING BEHAVIOR

Repeatable

SEEDED EXPERIMENTS

Inspectable

REQUEST-LEVEL EVIDENCE

A reproducible capacity experiment. The Overloaded and Optimized presets use the same 35 requests/min workload, 5-second mean service time, 60-minute arrival window, 5-second SLA, and seed 42. Only worker count changes.

OBSERVED METRIC	2 WORKERS	4 WORKERS
Completed requests	2,070	2,070
Theoretical utilization	145.8%	72.9%
Average waiting time	742.64 sec	1.92 sec
Average response time	747.57 sec	6.85 sec
SLA breach rate	99.95%	51.84%

What the experiment teaches. More capacity sharply reduces backlog, yet the 5-second SLA is still breached by over half the requests. Lower utilization alone does not guarantee a latency target.

Verification. These figures were generated by executing the current analysis code. The existing 51-test suite passed during verification. They are simulated outcomes from one seeded workload, not real-server benchmarks.

A small, local simulation project that makes the relationship between demand, capacity, and waiting tangible—and keeps the limits of its model explicit.